PostgreSQL 16
Streaming Replication — Database Restore Configuration Guide

Comprehensive parameter tuning for 200 GB, 300 GB, 500 GB, 700 GB, 1 TB, and 2 TB database restores
with Primary–Secondary streaming replication topology

Document Version	2.0
PostgreSQL Version	16.x
Replication Type	Physical Streaming Replication (Primary → Secondary)
Main Filesystem	`/apps`
Data Directory	`/apps/pgsql_data/16`
Config Files Location	`/apps/pgsql_data/16/postgresql.conf & postgresql.auto.conf`
WAL Directory	`/apps/pgsql_data/16/pg_wal`
Archive Directory	`/apps/pgsql_archives`
Logs Directory	`/apps/logs`
Binaries Directory	`/apps/bin`
Backups Directory	`/apps/backups` (pgbackrest, pg_basebackup, pg_dumpall dumps)
Date	March 14, 2026
Classification	Internal — DBA Team

Environment Directory Layout

```
/apps/ (main filesystem)
■■■ pgsql_data/
■ ■■■ 13/ (PostgreSQL 13 data - source for migration)
■ ■■■ 14/ (PostgreSQL 14 data - source for migration)
■ ■■■ 15/ (PostgreSQL 15 data - source for migration)
■ ■■■ 16/ (PostgreSQL 16 data directory)
■ ■ ■■■ postgresql.conf
■ ■ ■■■ postgresql.auto.conf
■ ■ ■■■ pg_wal/
■ ■ ■■■ pg_replslot/
■ ■ ■■■ base/
■ ■ ■■■ global/
■ ■■■ 17/ (PostgreSQL 17 data directory)
■ ■■■ 18/ (PostgreSQL 18 data directory)
■■■ pgsql_archives/ (WAL archive destination)
■■■ logs/ (PostgreSQL server logs)
■■■ bin/ (PostgreSQL binaries/utilities)
■■■ backups/ (pgbackrest, pg_basebackup, pg_dumpall dumps)
```

Table of Contents

1. Overview and Architecture

This document provides detailed PostgreSQL configuration parameters that must be tuned before restoring large databases (200 GB through 2 TB) in a streaming replication environment. It also covers full cluster migration using `pg_dumpall` from PostgreSQL 13, 14, and 15 to target versions 16, 17, and 18. The topology consists of:

- **Primary Server** — Accepts all read/write operations; source of WAL stream
- **Secondary Server (Standby)** — Receives and replays WAL; read-only queries

All configuration files reside in the data directory at `/apps/pgsql_data/16/`. Both `postgresql.conf` and `postgresql.auto.conf` are located in this directory. The WAL segment files are stored in `/apps/pgsql_data/16/pg_wal/` which is part of the same data directory. WAL archives are written to `/apps/pgsql_archives/`, server logs are in `/apps/logs/`, and all backups (pgbackrest, pg_basebackup, and other utilities) are stored in `/apps/backups/`. All directories reside under the `/apps` main filesystem.

During a large database restore (e.g., migrating from PostgreSQL 13 to 16, or refreshing a database), the Primary generates an enormous volume of Write-Ahead Log (WAL). If configuration parameters are not tuned appropriately, the following failures will occur:

- Replication slot invalidation due to WAL exceeding `max_slot_wal_keep_size`
- Disk exhaustion on Primary's `/apps/pgsql_data/16/pg_wal/` directory due to retained WAL segments
- Excessive checkpointing (every few seconds) degrading I/O performance and restore speed
- Standby disconnection and inability to reconnect (requested WAL segment already removed)
- OOM (Out of Memory) kills during FK creation and index builds on large tables
- Transaction ID wraparound risk if autovacuum remains off too long post-restore

2. Why Configuration Changes Are Needed During Restore

Parameter	Why It Needs to Change During Restore
`shared_buffers`	Larger buffer pool reduces disk I/O during bulk inserts and index creation. Needs to scale with database size.
`work_mem`	FK validation, index sorts, and hash joins during constraint creation need more per-operation memory to avoid disk-based sorts.
`maintenance_work_mem`	Directly controls memory for CREATE INDEX, ALTER TABLE ADD CONSTRAINT, VACUUM, and CLUSTER. Largest single impact on restore speed.

max_wal_size	During bulk operations, WAL is generated at extreme rates. A small max_wal_size forces checkpoints every few seconds, destroying I/O throughput.
checkpoint_timeout	Works with max_wal_size to control checkpoint frequency. Must be increased so time-based checkpoints don't trigger during heavy WAL generation.
checkpoint_completion_target	Spreads checkpoint I/O over a longer period, preventing I/O spikes that starve the WAL writer and replication.
max_slot_wal_keep_size	Controls how much WAL the replication slot retains. Too low = slot invalidation; too high = disk full risk.
wal_keep_size	Belt-and-suspenders WAL retention independent of slots. Ensures WAL files are kept even if slot logic has edge cases.
wal_buffers	In-memory buffer for WAL before flushing to disk. Larger buffers reduce WAL write contention during high-throughput operations.
autovacuum	Must be OFF during restore to avoid autovacuum competing for I/O and generating additional WAL during bulk loads.
full_page_writes	Can be temporarily OFF during initial data load (NOT during FK/index phase) to reduce WAL volume by ~50%. Must be ON for crash safety.
max_parallel_maintenance_workers	Controls parallel CREATE INDEX. Increasing this speeds up index creation significantly on multi-core systems.
wal_sender_timeout	Default 60s too aggressive during restore. Standby replay falls behind and the sender disconnects prematurely.
wal_receiver_timeout	On Secondary, controls how long to wait before declaring connection lost. Must match sender timeout increase.
effective_io_concurrency	For SSD/NVMe storage, increasing this allows PostgreSQL to issue more concurrent I/O requests during index builds.

3. Server Change Order — Primary First vs Secondary First

CRITICAL: The order in which you apply configuration changes matters. Incorrect ordering can cause replication breaks, connection failures, or data inconsistency.

3.1 Changes That Must Be Applied to SECONDARY FIRST

These parameters affect how the Secondary receives and processes WAL. Applying them to the Secondary first ensures it is ready to handle the increased load before the Primary starts generating it.

Parameter	Reason for Secondary First
wal_receiver_timeout	Secondary must tolerate longer gaps before Primary increases sender timeout
shared_buffers *	Secondary needs memory to replay WAL efficiently; requires restart
work_mem	Secondary applies WAL that includes sort/hash operations
maintenance_work_mem	Secondary replays index creation WAL and needs matching memory
wal_buffers *	Secondary writes WAL during recovery; needs larger buffers; requires restart
max_parallel_maintenance_workers	Secondary must match Primary for parallel replay capabilities
hot_standby_feedback	Must be ON so Primary knows standby's xmin and defers vacuum

*** Parameters marked with asterisk (*) require a PostgreSQL restart, not just reload. Config files are at `/apps/pgsql_data/16/postgresql.auto.conf`**

3.2 Changes That Must Be Applied to PRIMARY (After Secondary Is Ready)

Parameter	Reason for Primary Application (after Secondary is ready)
max_wal_size	Controls checkpoint frequency; Primary generates WAL
checkpoint_timeout	Controls time-based checkpoint trigger on Primary
checkpoint_completion_target	Controls checkpoint I/O spreading on Primary
max_slot_wal_keep_size	Controls slot invalidation threshold on Primary
wal_keep_size	Controls WAL retention on Primary's pg_wal directory
wal_sender_timeout	Primary's walsender tolerance; apply after Secondary is ready
autovacuum = off	Disable on Primary during restore; Secondary inherits via WAL
full_page_writes	Only on Primary; Secondary follows Primary's WAL content
effective_io_concurrency	Primary's I/O parallelism for index builds

3.3 Summary — Implementation Order

Order	Server	Action	Restart?
Step 1	SECONDARY	Apply memory params (shared_buffers, work_mem, maintenance_work_mem, wal_buffers)	YES
Step 2	SECONDARY	Apply timeout params (wal_receiver_timeout, hot_standby_feedback)	NO — Reload
Step 3	SECONDARY	Restart Secondary and verify streaming resumed	RESTART
Step 4	PRIMARY	Apply memory params (shared_buffers, work_mem, maintenance_work_mem, wal_buffers)	YES
Step 5	PRIMARY	Apply WAL/checkpoint params (max_wal_size, checkpoint_timeout, etc.)	NO — Reload
Step 6	PRIMARY	Apply replication params (max_slot_wal_keep_size, wal_keep_size, wal_sender_timeout)	NO — Reload
Step 7	PRIMARY	Disable autovacuum, optionally disable full_page_writes	NO — Reload
Step 8	PRIMARY	Restart Primary for shared_buffers/wal_buffers changes	RESTART
Step 9	BOTH	Verify replication is healthy before starting restore	VERIFY

4. Master Parameter Reference — All Database Sizes

The following master table shows recommended parameter values for each database size. Values below are for the **RESTORE/MIGRATION window ONLY**. Production values must be restored after migration completes (see Section 12).

IMPORTANT: All configuration changes are written to `/apps/pgsql_data/16/postgresql.auto.conf` via ALTER SYSTEM. The pg_wal directory is at `/apps/pgsql_data/16/pg_wal/`. Ensure sufficient free disk space on the filesystem hosting this directory.

Parameter	Server	200 GB	300 GB	500 GB	700 GB	1 TB	2 TB
shared_buffers *	BOTH	16 GB	24 GB	24 GB	24 GB	32 GB	32 GB
work_mem	BOTH	40 MB	60 MB	80 MB	100 MB	128 MB	192 MB
maintenance_work_mem	BOTH	2 GB	4 GB	4 GB	6 GB	8 GB	10 GB
max_wal_size	PRI	32 GB	48 GB	64 GB	96 GB	128 GB	192 GB
checkpoint_timeout	PRI	30 min	45 min	60 min	60 min	60 min	60 min
chkpt_completion_target	PRI	0.9	0.9	0.9	0.9	0.9	0.9
max_slot_wal_keep_size	PRI	40 GB	60 GB	80 GB	120 GB	200 GB	400 GB
wal_keep_size	PRI	20 GB	30 GB	40 GB	60 GB	100 GB	200 GB
wal_buffers *	BOTH	256 MB	256 MB	512 MB	512 MB	1 GB	1 GB
autovacuum	PRI	off	off	off	off	off	off
full_page_writes	PRI	off **	off **	off **	off **	off **	off **
max_parallel_maint_wk	BOTH	4	4	6	8	8	12
max_parallel_wk_gather	BOTH	2	2	2	2	4	4
effective_io_concurrency	BOTH	200	200	200	200	200	200
wal_sender_timeout	PRI	300s	300s	600s	600s	900s	1200s

Parameter	Server	200 GB	300 GB	500 GB	700 GB	1 TB	2 TB
wal_receiver_timeout	SEC	300s	300s	600s	600s	900s	1200s
hot_standby_feedback	SEC	on	on	on	on	on	on

*** Requires PostgreSQL restart. ** full_page_writes=off ONLY during initial data load. Must be ON before FK/index creation. PRI=Primary, SEC=Secondary.**

4.1 Disk Space Requirements for pg_wal Directory

Database Size	Est. WAL Generated	Min Free in pg_wal	Recommended Free	Est. Restore Time
200 GB	80–120 GB	120 GB	180 GB	2–5 hours
300 GB	120–200 GB	200 GB	300 GB	4–8 hours
500 GB	200–400 GB	350 GB	500 GB	8–16 hours
700 GB	300–600 GB	500 GB	750 GB	12–24 hours
1 TB	500–900 GB	800 GB	1.2 TB	18–36 hours
2 TB	1–2 TB	1.5 TB	2.5 TB	36–72 hours

NOTE: These estimates assume pg_wal is on the same filesystem as the data directory (`/apps/pgsql_data/16`). Since all directories (data, WAL, archives, logs, backups) reside under `/apps`, free space is shared. Account for total usage across `/apps/pgsql_archives`, `/apps/backups`, and `/apps/logs` when calculating available space. For 1 TB+ restores, consider dropping the replication slot entirely (see Section 15).

5. Detailed Configuration — 200 GB Database Restore

Assumed Server Spec	32–64 GB RAM, 8+ CPU cores
Estimated WAL Generation	~80–120 GB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 120 GB free
Estimated Restore Duration	2–5 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**16GB**	Both	YES
`work_mem`	**40MB**	Both	NO
`maintenance_work_mem`	**2GB**	Both	NO
`max_wal_size`	**32GB**	Primary	NO
`checkpoint_timeout`	**'30min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**40GB**	Primary	NO
`wal_keep_size`	**20GB**	Primary	NO
`wal_buffers`	**256MB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**4**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**300s**	Primary	NO
`wal_receiver_timeout`	**300s**	Secondary	NO
`hot_standby_feedback`	**on**	Secondary	NO

5.1 ALTER SYSTEM Commands — 200 GB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '16GB';
ALTER SYSTEM SET work_mem = '40MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '300s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '16GB';
ALTER SYSTEM SET work_mem = '40MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET max_wal_size = '32GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '40GB';
ALTER SYSTEM SET wal_keep_size = '20GB';
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '300s';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

6. Detailed Configuration — 300 GB Database Restore

Assumed Server Spec	64 GB RAM, 8–16 CPU cores
Estimated WAL Generation	~120–200 GB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 200 GB free
Estimated Restore Duration	4–8 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**24GB**	Both	YES
`work_mem`	**60MB**	Both	NO
`maintenance_work_mem`	**4GB**	Both	NO
`max_wal_size`	**48GB**	Primary	NO
`checkpoint_timeout`	**'45min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**60GB**	Primary	NO
`wal_keep_size`	**30GB**	Primary	NO
`wal_buffers`	**256MB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**4**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**300s**	Primary	NO
`wal_receiver_timeout`	**300s**	Secondary	NO
`hot_standby_feedback`	**on**	Secondary	NO

6.1 ALTER SYSTEM Commands — 300 GB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '60MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '300s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '60MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET max_wal_size = '48GB';
ALTER SYSTEM SET checkpoint_timeout = '45min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '60GB';
ALTER SYSTEM SET wal_keep_size = '30GB';
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '300s';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

7. Detailed Configuration — 500 GB Database Restore

Assumed Server Spec	64–128 GB RAM, 16+ CPU cores
Estimated WAL Generation	~200–400 GB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 350 GB free
Estimated Restore Duration	8–16 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**24GB**	Both	YES
`work_mem`	**80MB**	Both	NO
`maintenance_work_mem`	**4GB**	Both	NO
`max_wal_size`	**64GB**	Primary	NO
`checkpoint_timeout`	**'60min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**80GB**	Primary	NO
`wal_keep_size`	**40GB**	Primary	NO
`wal_buffers`	**512MB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**6**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**600s**	Primary	NO
`wal_receiver_timeout`	**600s**	Secondary	NO
`hot_standby_feedback`	**on**	Secondary	NO

7.1 ALTER SYSTEM Commands — 500 GB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '80MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET wal_buffers = '512MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '6';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '600s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '80MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET max_wal_size = '64GB';
ALTER SYSTEM SET checkpoint_timeout = '60min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '80GB';
ALTER SYSTEM SET wal_keep_size = '40GB';
ALTER SYSTEM SET wal_buffers = '512MB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '6';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '600s';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

8. Detailed Configuration — 700 GB Database Restore

Assumed Server Spec	128 GB RAM, 16–32 CPU cores
Estimated WAL Generation	~300–600 GB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 500 GB free
Estimated Restore Duration	12–24 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**24GB**	Both	YES
`work_mem`	**100MB**	Both	NO
`maintenance_work_mem`	**6GB**	Both	NO
`max_wal_size`	**96GB**	Primary	NO
`checkpoint_timeout`	**'60min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**120GB**	Primary	NO
`wal_keep_size`	**60GB**	Primary	NO
`wal_buffers`	**512MB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**8**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**600s**	Primary	NO
`wal_receiver_timeout`	**600s**	Secondary	NO
`hot_standby_feedback`	**on**	Secondary	NO

8.1 ALTER SYSTEM Commands — 700 GB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '100MB';
ALTER SYSTEM SET maintenance_work_mem = '6GB';
ALTER SYSTEM SET wal_buffers = '512MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '8';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '600s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '100MB';
ALTER SYSTEM SET maintenance_work_mem = '6GB';
ALTER SYSTEM SET max_wal_size = '96GB';
ALTER SYSTEM SET checkpoint_timeout = '60min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '120GB';
ALTER SYSTEM SET wal_keep_size = '60GB';
ALTER SYSTEM SET wal_buffers = '512MB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '8';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '600s';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

9. Detailed Configuration — 1 TB Database Restore

STRONG RECOMMENDATION for 1 TB: For databases this large, the safest approach is to DROP the replication slot before restore and rebuild the Secondary afterwards using pg_basebackup. This eliminates all risk of slot invalidation and reduces restore time by 15–30%. See Section 15.3 for the detailed procedure.

Assumed Server Spec	128–256 GB RAM, 32+ CPU cores
Estimated WAL Generation	~500–900 GB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 800 GB free
Estimated Restore Duration	18–36 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**32GB**	Both	YES
`work_mem`	**128MB**	Both	NO
`maintenance_work_mem`	**8GB**	Both	NO
`max_wal_size`	**128GB**	Primary	NO
`checkpoint_timeout`	**'60min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**200GB**	Primary	NO
`wal_keep_size`	**100GB**	Primary	NO
`wal_buffers`	**1GB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**8**	Both	NO
`max_parallel_workers_per_gather`	**4**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**900s**	Primary	NO

Parameter	Value	Server	Restart?
wal_receiver_timeout	**900s**	Secondary	NO
hot_standby_feedback	**on**	Secondary	NO

9.1 ALTER SYSTEM Commands — 1 TB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '32GB';
ALTER SYSTEM SET work_mem = '128MB';
ALTER SYSTEM SET maintenance_work_mem = '8GB';
ALTER SYSTEM SET wal_buffers = '1GB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '8';
ALTER SYSTEM SET max_parallel_workers_per_gather = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '900s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '32GB';
ALTER SYSTEM SET work_mem = '128MB';
ALTER SYSTEM SET maintenance_work_mem = '8GB';
ALTER SYSTEM SET max_wal_size = '128GB';
ALTER SYSTEM SET checkpoint_timeout = '60min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '200GB';
ALTER SYSTEM SET wal_keep_size = '100GB';
ALTER SYSTEM SET wal_buffers = '1GB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '8';
ALTER SYSTEM SET max_parallel_workers_per_gather = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '900s';
```

```
SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

10. Detailed Configuration — 2 TB Database Restore

STRONG RECOMMENDATION for 2 TB: For databases this large, the safest approach is to DROP the replication slot before restore and rebuild the Secondary afterwards using pg_basebackup. This eliminates all risk of slot invalidation and reduces restore time by 15–30%. See Section 15.3 for the detailed procedure.

Assumed Server Spec	256–512 GB RAM, 48–64+ CPU cores
Estimated WAL Generation	~1–2 TB of WAL
Disk Space Needed (/apps/pgsql_data/16/pg_wal)	At least 1.5 TB free
Estimated Restore Duration	36–72 hours
Config File	`/apps/pgsql_data/16/postgresql.auto.conf`

Parameter	Value	Server	Restart?
`shared_buffers`	**32GB**	Both	YES
`work_mem`	**192MB**	Both	NO
`maintenance_work_mem`	**10GB**	Both	NO
`max_wal_size`	**192GB**	Primary	NO
`checkpoint_timeout`	**'60min'**	Primary	NO
`checkpoint_completion_target`	**0.9**	Primary	NO
`max_slot_wal_keep_size`	**400GB**	Primary	NO
`wal_keep_size`	**200GB**	Primary	NO
`wal_buffers`	**1GB**	Both	YES
`autovacuum`	**off**	Primary	NO
`full_page_writes`	**off**	Primary	NO
`max_parallel_maintenance_workers`	**12**	Both	NO
`max_parallel_workers_per_gather`	**4**	Both	NO
`effective_io_concurrency`	**200**	Both	NO
`wal_sender_timeout`	**1200s**	Primary	NO

Parameter	Value	Server	Restart?
wal_receiver_timeout	**1200s**	Secondary	NO
hot_standby_feedback	**on**	Secondary	NO

10.1 ALTER SYSTEM Commands — 2 TB

On SECONDARY (apply FIRST):

```
-- Connect to Secondary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '32GB';
ALTER SYSTEM SET work_mem = '192MB';
ALTER SYSTEM SET maintenance_work_mem = '10GB';
ALTER SYSTEM SET wal_buffers = '1GB';
ALTER SYSTEM SET max_parallel_maintenance_workers = '12';
ALTER SYSTEM SET max_parallel_workers_per_gather = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_receiver_timeout = '1200s';
ALTER SYSTEM SET hot_standby_feedback = 'on';

SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

On PRIMARY (apply AFTER Secondary is streaming):

```
-- Connect to Primary: sudo -u postgres psql
-- Config written to: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET shared_buffers = '32GB';
ALTER SYSTEM SET work_mem = '192MB';
ALTER SYSTEM SET maintenance_work_mem = '10GB';
ALTER SYSTEM SET max_wal_size = '192GB';
ALTER SYSTEM SET checkpoint_timeout = '60min';
ALTER SYSTEM SET checkpoint_completion_target = '0.9';
ALTER SYSTEM SET max_slot_wal_keep_size = '400GB';
ALTER SYSTEM SET wal_keep_size = '200GB';
ALTER SYSTEM SET wal_buffers = '1GB';
ALTER SYSTEM SET autovacuum = 'off';
ALTER SYSTEM SET full_page_writes = 'off';
ALTER SYSTEM SET max_parallel_maintenance_workers = '12';
ALTER SYSTEM SET max_parallel_workers_per_gather = '4';
ALTER SYSTEM SET effective_io_concurrency = '200';
ALTER SYSTEM SET wal_sender_timeout = '1200s';
```

```
SELECT pg_reload_conf();

-- shared_buffers and wal_buffers require RESTART:
-- sudo systemctl restart postgresql
-- OR: pg_ctl -D /apps/pgsql_data/16 restart
```

11. Step-by-Step Implementation Procedure

Follow these steps exactly in order. Do not skip any step. Each step includes the exact commands to run. Replace parameter values with the appropriate values for your database size from Sections 5–10.

Step 1: Pre-Flight Checks on BOTH Servers

Verify current replication state and record existing parameter values.

```
-- On PRIMARY — check replication status:
SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;

-- Check slot health:
SELECT slot_name, active, restart_lsn, wal_status, conflicting
FROM pg_replication_slots;

-- On BOTH — record current parameter values:
SHOW shared_buffers; SHOW work_mem; SHOW maintenance_work_mem;
SHOW max_wal_size; SHOW checkpoint_timeout; SHOW max_slot_wal_keep_size;
SHOW autovacuum; SHOW wal_sender_timeout; SHOW wal_receiver_timeout;

-- SAVE ALL OUTPUT. Needed for rollback in Section 12.
```

Expected Result: Replication state = 'streaming'. Slot wal_status = 'reserved'. Record all values.

Step 2: Backup Current Configuration on BOTH Servers

Create backups before making any changes.

```
-- Run on BOTH servers:
cp /apps/pgsql_data/16/postgresql.auto.conf
/apps/pgsql_data/16/postgresql.auto.conf.pre_restore_backup
cp /apps/pgsql_data/16/postgresql.conf
/apps/pgsql_data/16/postgresql.conf.pre_restore_backup

-- Also keep a copy in the backups directory:
cp /apps/pgsql_data/16/postgresql.auto.conf
/apps/backups/postgresql.auto.conf.pre_restore_backup
cp /apps/pgsql_data/16/postgresql.conf
/apps/backups/postgresql.conf.pre_restore_backup

-- Also export settings from PostgreSQL:
COPY (
SELECT name, setting, unit, source FROM pg_settings
WHERE source != 'default' ORDER BY name
) TO '/apps/backups/pg_settings_backup.csv' WITH CSV HEADER;
```

Expected Result: Backup files created at /apps/pgsql_data/16/postgresql.auto.conf.pre_restore_backup and /apps/backups/

Step 3: Check Disk Space on BOTH Servers

Ensure sufficient disk space on the filesystem hosting /apps/pgsql_data/16/pg_wal.

```
-- Check current WAL directory size:
SELECT pg_size_pretty(sum(size)) AS current_wal_size FROM pg_ls_waldir();

-- Check filesystem free space (run in bash, not psql):
df -h /apps # Main filesystem
df -h /apps/pgsql_data/16/pg_wal
df -h /apps/pgsql_data/16
df -h /apps/pgsql_archives
df -h /apps/backups

-- Check existing backup sizes in /apps/backups:
du -sh /apps/backups/*

-- Minimum free space (in pg_wal filesystem):
-- 200 GB restore: 120 GB | 300 GB restore: 200 GB
-- 500 GB restore: 350 GB | 700 GB restore: 500 GB
-- 1 TB restore: 800 GB | 2 TB restore: 1.5 TB

-- NOTE: Since pg_wal, data, archives, logs, and backups all
-- reside under /apps, free space is SHARED across all.
-- Account for total usage across ALL directories.
```

Expected Result: If insufficient space, expand filesystem or consider dropping slot (Section 15).

Step 4: Apply Configuration on SECONDARY Server

Apply memory and timeout parameters to the Secondary FIRST.

```
-- Connect to Secondary as superuser:
sudo -u postgres psql

-- Apply parameters (example for 300 GB — adjust per your size):
ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '60MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = 4;
ALTER SYSTEM SET effective_io_concurrency = 200;
ALTER SYSTEM SET wal_receiver_timeout = '300s';
ALTER SYSTEM SET hot_standby_feedback = on;

-- Verify changes are written to /apps/pgsql_data/16/postgresql.auto.conf:
\! cat /apps/pgsql_data/16/postgresql.auto.conf
```

Expected Result: Changes written but NOT active until restart.

Step 5: Restart SECONDARY Server

shared_buffers and wal_buffers require a full PostgreSQL restart.

```
-- From OS command line (not psql):
sudo systemctl restart postgresql
# OR: sudo -u postgres pg_ctl -D /apps/pgsql_data/16 restart

-- Wait 15 seconds, then verify:
sudo systemctl status postgresql

-- Verify parameters took effect:
sudo -u postgres psql -c "SHOW shared_buffers;"
sudo -u postgres psql -c "SHOW maintenance_work_mem;"
sudo -u postgres psql -c "SHOW wal_buffers;"
sudo -u postgres psql -c "SHOW wal_receiver_timeout;"

-- Verify replication resumed:
sudo -u postgres psql -c "SELECT status, received_lsn, latest_end_lsn FROM
pg_stat_wal_receiver;"
```

Expected Result: Parameters show new values. pg_stat_wal_receiver shows status='streaming'.

Step 6: Verify Secondary is Healthy Before Proceeding

Do NOT proceed to Primary changes until Secondary is confirmed healthy.

```
-- On PRIMARY, verify Secondary reconnected:
SELECT client_addr, state, sent_lsn, replay_lsn,
pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes
FROM pg_stat_replication;

-- Expected: state='streaming', lag_bytes < 1 MB
-- If lag is large, WAIT for it to decrease before proceeding.
```

Expected Result: STOP HERE if Secondary is not streaming. Fix the issue before touching Primary.

Step 7: Apply Configuration on PRIMARY Server

Apply all parameters to the Primary.

```
sudo -u postgres psql

-- Memory parameters (example for 300 GB):
ALTER SYSTEM SET shared_buffers = '24GB';
ALTER SYSTEM SET work_mem = '60MB';
ALTER SYSTEM SET maintenance_work_mem = '4GB';
```

```
ALTER SYSTEM SET wal_buffers = '256MB';

-- WAL/checkpoint parameters:
ALTER SYSTEM SET max_wal_size = '48GB';
ALTER SYSTEM SET checkpoint_timeout = '45min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;

-- Replication parameters:
ALTER SYSTEM SET max_slot_wal_keep_size = '60GB';
ALTER SYSTEM SET wal_keep_size = '30GB';
ALTER SYSTEM SET wal_sender_timeout = '300s';

-- Restore-specific parameters:
ALTER SYSTEM SET autovacuum = off;
ALTER SYSTEM SET max_parallel_maintenance_workers = 4;
ALTER SYSTEM SET effective_io_concurrency = 200;

-- Verify: \! cat /apps/pgsql_data/16/postgresql.auto.conf
```

Expected Result: Changes written but not active until restart.

Step 8: Restart PRIMARY Server

Restart to activate shared_buffers and wal_buffers.

```
-- IMPORTANT: Brief Secondary disconnection will occur.

sudo systemctl restart postgresql
# OR: sudo -u postgres pg_ctl -D /apps/pgsql_data/16 restart

-- Wait 15 seconds, verify:
sudo systemctl status postgresql

-- Verify ALL parameters:
sudo -u postgres psql << 'EOF'
SHOW shared_buffers; SHOW work_mem; SHOW maintenance_work_mem;
SHOW max_wal_size; SHOW checkpoint_timeout;
SHOW max_slot_wal_keep_size; SHOW wal_keep_size;
SHOW autovacuum; SHOW wal_sender_timeout;
EOF
```

Expected Result: All parameters show new values.

Step 9: Final Replication Health Check

Verify everything before starting restore.

```
-- On PRIMARY:
SELECT client_addr, state, sent_lsn, replay_lsn FROM pg_stat_replication;
```

```
SELECT slot_name, active, restart_lsn, wal_status FROM pg_replication_slots;

-- Expected: state='streaming', wal_status='reserved', active=true

-- On SECONDARY:
SELECT status, received_lsn, latest_end_lsn FROM pg_stat_wal_receiver;
```

Expected Result: Both confirm healthy streaming. ONLY then proceed with restore.

Step 10: Begin the Database Restore

Start the actual restore operation on the PRIMARY.

```
-- Split restore into phases for better control:

-- Phase 1: Pre-data (schema without constraints):
pg_restore -h localhost -U postgres -d target_db \
--jobs=4 --no-owner --no-privileges \
--section=pre-data /apps/backups/dump.custom

-- Phase 2: Data load (full_page_writes can be OFF here):
pg_restore -h localhost -U postgres -d target_db \
--jobs=4 --no-owner --no-privileges \
--section=data /apps/backups/dump.custom

-- Phase 3: BEFORE post-data, re-enable full_page_writes:
ALTER SYSTEM SET full_page_writes = on;
SELECT pg_reload_conf();

-- Phase 4: Post-data (FK constraints, indexes — heaviest WAL):
pg_restore -h localhost -U postgres -d target_db \
--jobs=4 --no-owner --no-privileges \
--section=post-data /apps/backups/dump.custom

-- For 1 TB+ restores, increase --jobs to 6-8 if CPU allows
```

Expected Result: Post-data phase generates the most WAL. Monitor closely (Section 13).

12. Post-Restore: Reverting to Production Settings

CRITICAL: You MUST revert these parameters after restore completes. Running production with restore-tuned parameters causes: autovacuum=off → table bloat and transaction ID wraparound; full_page_writes=off → data corruption on crash; oversized WAL settings → wasted disk space.

12.1 Revert on PRIMARY First

```
-- On PRIMARY (connect: sudo -u postgres psql):
-- Config file: /apps/pgsql_data/16/postgresql.auto.conf

ALTER SYSTEM SET autovacuum = on; -- CRITICAL
ALTER SYSTEM SET full_page_writes = on; -- CRITICAL
ALTER SYSTEM SET max_wal_size = '16GB'; -- Production default
ALTER SYSTEM SET checkpoint_timeout = '15min';
ALTER SYSTEM SET max_slot_wal_keep_size = '20GB';
ALTER SYSTEM SET wal_keep_size = '2GB';
ALTER SYSTEM SET wal_sender_timeout = '60s';
ALTER SYSTEM SET work_mem = '16MB';
ALTER SYSTEM SET maintenance_work_mem = '1GB';
ALTER SYSTEM SET shared_buffers = '8GB'; -- Adjust per server RAM
ALTER SYSTEM SET wal_buffers = '64MB';
ALTER SYSTEM SET max_parallel_maintenance_workers = 2;
ALTER SYSTEM SET effective_io_concurrency = 200;
SELECT pg_reload_conf();

-- Restart PRIMARY for shared_buffers/wal_buffers change:
-- sudo systemctl restart postgresql
-- OR: sudo -u postgres pg_ctl -D /apps/pgsql_data/16 restart
```

12.2 Revert on SECONDARY

```
-- On SECONDARY:
ALTER SYSTEM SET shared_buffers = '8GB';
ALTER SYSTEM SET work_mem = '16MB';
ALTER SYSTEM SET maintenance_work_mem = '1GB';
ALTER SYSTEM SET wal_buffers = '64MB';
ALTER SYSTEM SET wal_receiver_timeout = '60s';
ALTER SYSTEM SET max_parallel_maintenance_workers = 2;
SELECT pg_reload_conf();

-- Restart SECONDARY:
-- sudo systemctl restart postgresql
-- OR: sudo -u postgres pg_ctl -D /apps/pgsql_data/16 restart
```

12.3 Run Full VACUUM ANALYZE

After re-enabling autovacuum, immediately run a manual VACUUM ANALYZE:

```
-- Single database:
VACUUM (VERBOSE, ANALYZE);

-- Or from command line with parallel jobs:
vacuumdb -h localhost -U postgres -d target_db --analyze --jobs=4 --verbose

-- For 1 TB+ databases, consider running VACUUM on individual large tables
first:
VACUUM (VERBOSE, ANALYZE) schema.large_table_1;
VACUUM (VERBOSE, ANALYZE) schema.large_table_2;
```

12.4 Verify Replication After Revert

```
-- On PRIMARY:
SELECT client_addr, state, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag
FROM pg_stat_replication;

SELECT slot_name, active, wal_status, conflicting FROM pg_replication_slots;
```

13. Monitoring During Restore

Run these queries every 5–10 minutes during restore. For 1 TB+ restores, consider automating these into a monitoring script.

13.1 Replication Lag

```
SELECT client_addr, state,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, write_lsn)) AS write_lag
FROM pg_stat_replication;

-- WARNING if replay_lag > 10 GB
-- CRITICAL if replay_lag > max_slot_wal_keep_size * 0.7
```

13.2 WAL Directory Size

```
SELECT pg_size_pretty(sum(size)) AS wal_dir_size FROM pg_ls_waldir();

-- OS level:
du -sh /apps/pgsql_data/16/pg_wal
df -h /apps/pgsql_data/16/pg_wal
df -h /apps # Overall /apps filesystem
du -sh /apps/pgsql_archives # WAL archives
du -sh /apps/backups # Backups (pgbackrest, pg_basebackup)
du -sh /apps/logs # Server logs

-- If WAL approaching 70% of free space on /apps: PAUSE RESTORE
```

13.3 Replication Slot Health

```
SELECT slot_name, active, wal_status, safe_wal_size,
conflicting, conflict_reason
FROM pg_replication_slots;

-- wal_status progression: reserved → extended → unreserved → lost
-- 'unreserved' = IMMEDIATE ACTION NEEDED
-- 'lost' or conflicting=true = SLOT IS BROKEN (see Section 15)
```

13.4 Checkpoint Frequency

```
-- Check PostgreSQL logs for:
-- 'checkpoints are occurring too frequently (N seconds apart)'
-- Log location: /apps/logs/
tail -100 /apps/logs/postgresql-*.log | grep -i checkpoint

-- If checkpoints < 2 min apart: increase max_wal_size
```

13.5 OS-Level Resource Monitoring

```
-- Disk I/O:
iostat -x 5 # Watch await and %util columns

-- Memory/Swap:
vmstat 5 # Watch si/so swap columns (should be 0)
free -h # Check available memory

-- OOM Check:
dmesg | grep -i 'out of memory'
journalctl -k | grep -i oom
```

13.6 Automated Monitoring Script (Optional)

```
#!/bin/bash
# Save as /apps/bin/monitor_restore.sh
# Run: watch -n 300 /apps/bin/monitor_restore.sh

echo '=== Replication Lag ==='
sudo -u postgres psql -c "SELECT client_addr,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag
FROM pg_stat_replication;"

echo '=== Slot Health ==='
sudo -u postgres psql -c "SELECT slot_name, wal_status, conflicting
FROM pg_replication_slots;"

echo '=== WAL Size ==='
du -sh /apps/pgsql_data/16/pg_wal

echo '=== /apps Filesystem Usage ==='
df -h /apps
echo "Archives: $(du -sh /apps/pgsql_archives | cut -f1)"
echo "Backups: $(du -sh /apps/backups | cut -f1)"
echo "Logs: $(du -sh /apps/logs | cut -f1)"
```

14. Troubleshooting Common Issues

Issue	Symptom	Root Cause	Resolution
Slot Invalidated	pg_replication_slots: conflicting=true, wal_status='lost'	WAL exceeded max_slot_wal_keep_size	Rebuild standby via pg_basebackup (Section 15.1)
Checkpoints every few seconds	Log: 'checkpoints occurring too frequently'	max_wal_size too small	Double max_wal_size and reload
Standby disconnects	Log: 'terminating walsender' or 'replication timeout'	wal_sender/receiver_timeout too short	Increase both to 600s+ and reload
Primary disk full	PostgreSQL stops writes. PANIC in logs.	WAL filling /apps filesystem	Emergency: DROP slot. Free space in archives/backups. Rebuild standby.
OOM kills PostgreSQL	dmesg: 'Out of memory: Kill process postgres'	shared_buffers + work_mem + maint_work_mem > RAM	Reduce maintenance_work_mem. Restart PostgreSQL.
Standby won't start	Log: 'could not map shared memory segment'	shared_buffers > kernel shmmax or system RAM	Reduce shared_buffers or: sysctl -w kernel.shmmax=VALUE
WAL segment removed	Error: 'requested WAL segment has already been removed'	Standby too far behind, WAL recycled	Rebuild standby with pg_basebackup
Restore extremely slow	pg_restore taking 3x expected time	Checkpoint storms or I/O contention	Check checkpoint freq, increase max_wal_size, check iostat
1 TB+ restore: constant slot warnings	Log: repeated 'slot exceeds limit' warnings	Even large max_slot_wal_keep_size insufficient	Drop slot, restore without replication, rebuild after

15. Emergency Procedures

15.1 Emergency: Replication Slot Broken During Restore

If the replication slot is invalidated during restore, the Primary data is safe. The Secondary must be rebuilt after the restore completes.

```
-- Step 1: Confirm slot is broken (on Primary):
SELECT slot_name, active, conflicting, conflict_reason, wal_status
FROM pg_replication_slots;

-- Step 2: Drop the broken slot (on Primary):
SELECT pg_drop_replication_slot('your_slot_name');

-- Step 3: Continue and COMPLETE the restore on Primary.
-- Step 4: After restore completes + VACUUM ANALYZE, rebuild Secondary:

-- On Secondary — stop PostgreSQL:
sudo systemctl stop postgresql

-- Remove old data directory:
sudo rm -rf /apps/pgsql_data/16/*

-- Take fresh base backup from Primary:
sudo -u postgres pg_basebackup -h PRIMARY_IP -U replication_user \
-D /apps/pgsql_data/16 -X stream -C -S your_slot_name -R -P

-- Start Secondary:
sudo systemctl start postgresql

-- Verify on Primary:
SELECT * FROM pg_stat_replication;
SELECT * FROM pg_replication_slots;
```

15.2 Emergency: Primary Disk Full

```
-- IMMEDIATE: Free space by dropping the replication slot:
SELECT pg_drop_replication_slot('your_slot_name');

-- Quick space recovery on /apps filesystem:
-- 1. Clean old WAL archives:
find /apps/pgsql_archives -name '*.gz' -mtime +2 -delete

-- 2. Clean old backups if safe to do so:
ls -lht /apps/backups/
-- Remove old/unneeded backup files to reclaim space

-- 3. Truncate or rotate large log files:
find /apps/logs -name '*.log' -size +500M -exec truncate -s 0 {} \;
```

```
-- If PostgreSQL is completely stuck and cannot accept SQL:
-- Manually clean old WAL files:
pg_archivecleanup /apps/pgsql_data/16/pg_wal

-- Or add emergency disk space and restart PostgreSQL
-- sudo systemctl restart postgresql
-- After recovery, rebuild standby from scratch
```

15.3 Recommended Strategy for 700 GB+ Restores: Drop Slot Before Restore

For very large restores (700 GB, 1 TB, 2 TB), the safest strategy is to intentionally disconnect replication before the restore and rebuild the Secondary afterwards. This eliminates ALL risk of slot invalidation and reduces restore time by 15–30%.

```
-- ■■■ BEFORE RESTORE ■■■

-- Step 1: On Primary, note the slot name:
SELECT slot_name FROM pg_replication_slots;

-- Step 2: Drop the replication slot:
SELECT pg_drop_replication_slot('your_slot_name');

-- Step 3: Optionally set wal_keep_size as safety net:
ALTER SYSTEM SET wal_keep_size = '200GB'; -- for 1 TB restore
SELECT pg_reload_conf();

-- Step 4: Stop the Secondary to save resources:
-- (on Secondary) sudo systemctl stop postgresql

-- Step 5: Run the entire restore on Primary
-- (No replication overhead = faster restore)

-- ■■■ AFTER RESTORE COMPLETES ■■■

-- Step 6: Revert production parameters on Primary (Section 12)
-- Step 7: Run VACUUM ANALYZE on Primary

-- Step 8: Rebuild Secondary with pg_basebackup:
-- (on Secondary):
sudo systemctl stop postgresql
sudo rm -rf /apps/pgsql_data/16/*
sudo -u postgres pg_basebackup -h PRIMARY_IP -U replication_user \
-D /apps/pgsql_data/16 -X stream -C -S your_slot_name -R -P
sudo systemctl start postgresql

-- Step 9: Verify replication on Primary:
SELECT * FROM pg_stat_replication;
```

```
SELECT * FROM pg_replication_slots;
```

TRADE-OFF: No standby availability during the restore window. For databases 1 TB+, this is almost always the correct choice — trying to keep replication alive during a multi-day restore is fragile and risky.

16. pg_dumpall Cluster Migration — Overview and Version Compatibility

`pg_dumpall` is the standard PostgreSQL utility for backing up an entire cluster — all databases, roles, tablespaces, and global objects — into a single SQL script. It is the primary tool for performing major version upgrades when `pg_upgrade` is not suitable or when migrating across different server hardware. This section covers migration from PostgreSQL 13, 14, and 15 to target versions 16, 17, and 18.

16.1 What pg_dumpall Backs Up

- All databases in the cluster (schema + data for each)
- All roles (users and groups) with passwords and attributes
- All tablespace definitions
- All database-level GRANTs and role memberships
- Database configuration settings set via ALTER DATABASE ... SET
- Role-level configuration settings set via ALTER ROLE ... SET

16.2 What pg_dumpall Does NOT Back Up

- postgresql.conf and postgresql.auto.conf settings (must be manually migrated)
- pg_hba.conf authentication rules (must be manually migrated)
- Replication slots and replication configuration
- WAL files and WAL archives
- Physical file locations (tablespace paths may need adjustment)
- Extensions binaries (must be installed on target before restore)
- Foreign Data Wrapper connections and server definitions (included in dump but credentials may need updating)

16.3 Version Compatibility Matrix

The following matrix shows supported migration paths. **CRITICAL RULE:** Always use the `pg_dumpall` binary from the **TARGET** (newer) version to connect to the SOURCE (older) server. The newer version's pg_dumpall understands older formats, but the reverse is not true.

Source Version	Target Version	Supported?	pg_dumpall Binary to Use	Key Considerations
PostgreSQL 13	PostgreSQL 16	YES	PG 16 pg_dumpall	3 major versions jump. Review deprecated features.

Source Version	Target Version	Supported?	pg_dumpall Binary to Use	Key Considerations
PostgreSQL 13	PostgreSQL 17	YES	PG 17 pg_dumpall	4 major versions. Significant syntax/behavior changes.
PostgreSQL 13	PostgreSQL 18	YES	PG 18 pg_dumpall	5 major versions. Thorough testing essential.
PostgreSQL 14	PostgreSQL 16	YES	PG 16 pg_dumpall	2 major versions. Relatively smooth upgrade.
PostgreSQL 14	PostgreSQL 17	YES	PG 17 pg_dumpall	3 major versions. Review breaking changes.
PostgreSQL 14	PostgreSQL 18	YES	PG 18 pg_dumpall	4 major versions. Check extension compatibility.
PostgreSQL 15	PostgreSQL 16	YES	PG 16 pg_dumpall	1 major version. Minimal breaking changes.
PostgreSQL 15	PostgreSQL 17	YES	PG 17 pg_dumpall	2 major versions. Check public schema changes.
PostgreSQL 15	PostgreSQL 18	YES	PG 18 pg_dumpall	3 major versions. Review release notes.

16.4 pg_dumpall vs pg_dump vs pg_upgrade — When to Use What

Method	Scope	Downtime	Best For	Limitations
pg_dumpall	Entire cluster (all DBs, roles)	HIGH — full dump + restore	Cross-version migration, cross-platform, cross-hardware	Slowest method. Dump size = DB size. Long downtime for large clusters.
pg_dump / pg_restore	Single database	MODERATE — per-database	Migrating specific databases, parallel restore with --jobs	Does NOT dump roles or global objects. Must use pg_dumpall --globals-only separately.
pg_upgrade	Entire cluster in-place	LOW — minutes to hours	Same-hardware major version upgrade with minimal downtime	Requires both old and new binaries. Cannot cross hardware. Link mode requires same filesystem.

RECOMMENDATION: For large clusters (500 GB+) where downtime is a concern, prefer the hybrid approach: use `pg_dumpall --globals-only` for roles and global objects, then use `pg_dump -Fc` (custom format) + `pg_restore --jobs=N` for each database individually. This allows parallel restore and significantly reduces total migration time.

17. pg_dumpall Backup from Source Cluster (PG 13 / 14 / 15)

This section provides step-by-step procedures for backing up the entire source cluster using pg_dumpall. These steps are performed on the **SOURCE server** (PostgreSQL 13, 14, or 15) before any changes are made to the target server.

17.1 Pre-Backup Checklist on Source Server

```
-- Step 1: Identify source PostgreSQL version:
psql -c "SELECT version();"

-- Step 2: List all databases and their sizes:
psql -c "SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database WHERE datistemplate = false ORDER BY pg_database_size(datname)
DESC;"

-- Step 3: List all roles:
psql -c "\du+"

-- Step 4: List all tablespaces:
psql -c "SELECT spcname, pg_tablespace_location(oid) FROM pg_tablespace;"

-- Step 5: List all installed extensions per database:
psql -c "SELECT datname FROM pg_database WHERE datistemplate = false;" -tA | \
while read db; do echo "=== $db ==="; psql -d $db -c "SELECT extname, extversion
FROM pg_extension;"; done

-- Step 6: Check total cluster size (this determines which config tier to use):
psql -c "SELECT pg_size_pretty(sum(pg_database_size(datname))) AS
total_cluster_size
FROM pg_database;"

-- Step 7: Check free disk space for dump file in /apps/backups:
df -h /apps/backups
-- Rule of thumb: Need 1x to 1.5x the total DB size for the dump file

-- Step 8: Record pg_hba.conf and postgresql.conf for manual migration:
cp /apps/pgsql_data/SOURCE_VERSION/pg_hba.conf
/apps/backups/pg_hba.conf.source_backup
cp /apps/pgsql_data/SOURCE_VERSION/postgresql.conf
/apps/backups/postgresql.conf.source_backup
cp /apps/pgsql_data/SOURCE_VERSION/postgresql.auto.conf
/apps/backups/postgresql.auto.conf.source_backup
```

IMPORTANT: Save the output of ALL the above queries. You will need them to verify the migration was successful (comparing database counts, sizes, role counts, extensions).

17.2 Method A: Full Cluster Dump Using pg_dumpall (Plain SQL)

This is the simplest approach — one command dumps everything. Use this for clusters under 200 GB where downtime is acceptable.

```
-- CRITICAL: Use the TARGET version's pg_dumpall binary, NOT the source's!
-- The target pg_dumpall connects to the source server remotely.

-- Example: Migrating from PG 13 to PG 16
-- Use PG 16's pg_dumpall to dump from PG 13 source server:

/usr/pgsql-16/bin/pg_dumpall \
-h SOURCE_SERVER_IP \
-p 5432 \
-U postgres \
--verbose \
--no-role-passwords \
-f /apps/backups/full_cluster_dump_pg13_to_pg16.sql \
2>&1 | tee /apps/logs/pg_dumpall_backup.log

-- To include role passwords (encrypted), remove --no-role-passwords:
/usr/pgsql-16/bin/pg_dumpall \
-h SOURCE_SERVER_IP \
-p 5432 \
-U postgres \
--verbose \
-f /apps/backups/full_cluster_dump_with_passwords.sql \
2>&1 | tee /apps/logs/pg_dumpall_backup.log

-- Check dump file size:
ls -lh /apps/backups/full_cluster_dump_pg13_to_pg16.sql
```

17.3 Method B: Hybrid Approach — Globals + Individual Database Dumps (Recommended for 200 GB+)

For large clusters, the hybrid approach is strongly recommended. It dumps global objects (roles, tablespaces) separately, then dumps each database in custom format which supports parallel restore via `pg_restore --jobs`.

```
-- ■■■ PHASE 1: Dump Global Objects (roles, tablespaces) ■■■
-- This is fast regardless of cluster size (seconds to dump)

-- Use TARGET version's pg_dumpall with --globals-only:
/usr/pgsql-16/bin/pg_dumpall \
-h SOURCE_SERVER_IP \
-p 5432 \
-U postgres \
--globals-only \
--verbose \
```

```
-f /apps/backups/globals_only.sql \
2>&1 | tee /apps/logs/pg_dumpall_globals.log


-- ■■■ PHASE 2: Dump Each Database in Custom Format ■■■
-- Custom format (-Fc) enables parallel restore with --jobs
-- Use TARGET version's pg_dump binary:

-- List databases to dump (exclude templates):
psql -h SOURCE_SERVER_IP -U postgres -tAc \
"SELECT datname FROM pg_database WHERE datistemplate = false AND datname !=
'postgres';"

-- Dump each database individually:
for DB_NAME in db1 db2 db3; do
echo "Dumping $DB_NAME ..."
/usr/pgsql-16/bin/pg_dump \
-h SOURCE_SERVER_IP \
-p 5432 \
-U postgres \
-Fc \
--verbose \
-d $DB_NAME \
-f /apps/backups/${DB_NAME}.dump \
2>&1 | tee /apps/logs/pg_dump_${DB_NAME}.log
done

-- Verify dump files:
ls -lh /apps/backups/*.dump
ls -lh /apps/backups/globals_only.sql
```

WHY HYBRID? A 500 GB plain-SQL pg_dumpall dump restores serially — one statement at a time. The hybrid approach with pg_dump -Fc + pg_restore --jobs=8 can be 4–6x faster for the data + index restoration phase. For a 1 TB cluster, this can save 12–24 hours of downtime.

17.4 Dump Time Estimates by Cluster Size

Cluster Size	pg_dumpall (Plain SQL)	pg_dump -Fc (Per DB)	Dump File Size (Approx.)
200 GB	30–60 minutes	20–45 minutes	40–80 GB (compressed in -Fc)
300 GB	45–90 minutes	30–60 minutes	60–120 GB
500 GB	1.5–3 hours	1–2 hours	100–200 GB
700 GB	2–4 hours	1.5–3 hours	140–280 GB
1 TB	3–6 hours	2–4 hours	200–400 GB

Cluster Size	pg_dumpall (Plain SQL)	pg_dump -Fc (Per DB)	Dump File Size (Approx.)
2 TB	6–12 hours	4–8 hours	400–800 GB

NOTE: Ensure `/apps/backups` has enough free space. Plain SQL dumps are roughly equal to database size. Custom format (-Fc) dumps are compressed and typically 40–60% smaller.

18. pg_dumpall Restore to Target Cluster (PG 16 / 17 / 18)

This section covers restoring the dump to a freshly initialized target PostgreSQL cluster. The target server must have PostgreSQL 16, 17, or 18 installed and initialized before proceeding. Apply the configuration tuning from Sections 5–10 BEFORE starting the restore.

18.1 Pre-Restore: Initialize and Prepare Target Cluster

```
-- Step 1: Initialize new cluster (if not already done):
-- For PG 16:
/usr/pgsql-16/bin/initdb -D /apps/pgsql_data/16 --encoding=UTF8
--locale=en_US.UTF-8
-- For PG 17:
/usr/pgsql-17/bin/initdb -D /apps/pgsql_data/17 --encoding=UTF8
--locale=en_US.UTF-8
-- For PG 18:
/usr/pgsql-18/bin/initdb -D /apps/pgsql_data/18 --encoding=UTF8
--locale=en_US.UTF-8

-- Step 2: Migrate pg_hba.conf from source (review and adjust):
cp /apps/backups/pg_hba.conf.source_backup
/apps/pgsql_data/TARGET_VERSION/pg_hba.conf
-- REVIEW the file — authentication methods may need updating for new version

-- Step 3: Install required extensions on target BEFORE restore:
-- Review extension list from pre-backup check (Step 5 in Section 17.1)
-- Install each required extension package, e.g.:
sudo yum install postgresql16-contrib # For PG 16
sudo yum install postgresql17-contrib # For PG 17
-- For third-party extensions: install matching version for target PG

-- Step 4: Start target PostgreSQL:
pg_ctl -D /apps/pgsql_data/TARGET_VERSION start -l
/apps/logs/postgresql_startup.log

-- Step 5: Apply restore-tuned configuration (from Sections 5-10):
-- Choose parameter values based on your total cluster size.
-- Apply the ALTER SYSTEM commands from the appropriate section.
-- Restart after applying shared_buffers and wal_buffers.
```

18.2 Restore Method A: Full pg_dumpall Plain SQL Restore

Use this if you performed a full `pg_dumpall` dump (Method A from Section 17.2). This restores everything in a single serial pass.

```
-- Restore the full cluster dump into the target:
-- Run on TARGET server:
```

```
psql -h localhost -p 5432 -U postgres \
-f /apps/backups/full_cluster_dump_pg13_to_pg16.sql \
-v ON_ERROR_STOP=0 \
2>&1 | tee /apps/logs/pg_dumpall_restore.log

-- NOTE: -v ON_ERROR_STOP=0 means continue on errors.
-- Some errors are EXPECTED during cross-version restore:
-- 'role postgres already exists' — harmless
-- 'database postgres already exists' — harmless
-- Extension version mismatch warnings — review case by case

-- Review the log for REAL errors:
grep -i 'ERROR' /apps/logs/pg_dumpall_restore.log | grep -v 'already exists'
```

18.3 Restore Method B: Hybrid — Globals First, Then Parallel Database Restore (Recommended)

Use this if you performed the hybrid dump (Method B from Section 17.3). This is significantly faster for large clusters due to parallel restore.

```
-- ■■■ PHASE 1: Restore Global Objects (roles, tablespaces) ■■■
-- This is fast — usually under 1 minute even for large clusters.

psql -h localhost -p 5432 -U postgres \
-f /apps/backups/globals_only.sql \
2>&1 | tee /apps/logs/restore_globals.log

-- Verify roles were created:
psql -c "\du+"


-- ■■■ PHASE 2: Create Empty Databases ■■■
-- Create each database before restoring into it:

for DB_NAME in db1 db2 db3; do
echo "Creating database $DB_NAME ..."
psql -h localhost -U postgres -c "CREATE DATABASE $DB_NAME;"
done


-- ■■■ PHASE 3: Parallel Restore Each Database ■■■
-- Use pg_restore with --jobs for parallel restore.
-- Adjust --jobs based on CPU cores (typically cores/2).

for DB_NAME in db1 db2 db3; do
echo "Restoring $DB_NAME ..."
/usr/pgsql-16/bin/pg_restore \
```

```
-h localhost \
-p 5432 \
-U postgres \
-d $DB_NAME \
--jobs=4 \
--verbose \
--no-owner \
--no-privileges \
/apps/backups/${DB_NAME}.dump \
2>&1 | tee /apps/logs/pg_restore_${DB_NAME}.log
echo "$DB_NAME restore complete."
done

-- Review errors for each database:
for f in /apps/logs/pg_restore_*.log; do
echo "=== $(basename $f) ==="
grep -c 'ERROR' $f
done
```

18.4 Restore Jobs Recommendation by Cluster Size

Cluster Size	--jobs for pg_restore	CPU Cores Assumed	Estimated Restore Time
200 GB	4	8+ cores	2–4 hours
300 GB	4–6	8–16 cores	3–6 hours
500 GB	6–8	16+ cores	6–12 hours
700 GB	8	16–32 cores	10–20 hours
1 TB	8–12	32+ cores	16–32 hours
2 TB	12–16	48–64 cores	30–60 hours

18.5 Post-Restore Verification on Target Cluster

```
-- Step 1: Compare database count and sizes with source:
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database WHERE datistemplate = false ORDER BY pg_database_size(datname)
DESC;

-- Step 2: Verify all roles exist:
\du+

-- Step 3: Verify extensions in each database:
-- Compare with pre-backup extension list from Section 17.1
psql -tAc "SELECT datname FROM pg_database WHERE datistemplate = false;" | \
while read db; do echo "=== $db ==="; psql -d $db -c "SELECT extname, extversion
```

```
FROM pg_extension;"; done

-- Step 4: Run ANALYZE to update optimizer statistics:
vacuumdb -h localhost -U postgres --all --analyze-only --jobs=4 --verbose

-- Step 5: Check for invalid indexes (can occur during migration):
SELECT schemaname, tablename, indexname, indexdef
FROM pg_indexes WHERE indexdef IS NULL;

-- Step 6: Reindex if needed (run per database):
-- REINDEX DATABASE db_name;

-- Step 7: Verify row counts on critical tables:
-- Compare with source: SELECT count(*) FROM schema.critical_table;
```

18.6 Set Up Streaming Replication After Migration

After the restore and verification are complete, set up streaming replication from the new Primary to the Secondary using pg_basebackup.

```
-- Step 1: On PRIMARY — ensure replication parameters are set:
ALTER SYSTEM SET wal_level = 'replica';
ALTER SYSTEM SET max_wal_senders = 5;
ALTER SYSTEM SET max_replication_slots = 5;
SELECT pg_reload_conf();

-- Step 2: On PRIMARY — create replication role (if not migrated):
CREATE ROLE replication_user WITH REPLICATION LOGIN PASSWORD 'secure_password';

-- Step 3: On PRIMARY — update pg_hba.conf for replication:
-- Add line: host replication replication_user SECONDARY_IP/32 scram-sha-256
-- Edit: /apps/pgsql_data/TARGET_VERSION/pg_hba.conf
SELECT pg_reload_conf();

-- Step 4: On PRIMARY — revert to production configuration (Section 12)
-- Step 5: Restart PRIMARY

-- Step 6: On SECONDARY — build standby from Primary:
sudo systemctl stop postgresql
sudo rm -rf /apps/pgsql_data/TARGET_VERSION/*

sudo -u postgres pg_basebackup \
-h PRIMARY_IP \
-U replication_user \
-D /apps/pgsql_data/TARGET_VERSION \
-X stream -C -S replication_slot_name -R -P

sudo systemctl start postgresql

-- Step 7: On PRIMARY — verify replication:
```

```
SELECT * FROM pg_stat_replication;
SELECT * FROM pg_replication_slots;
```

19. Version-Specific Migration Notes and Breaking Changes

Each major PostgreSQL version introduces changes that may affect your migration. Review the relevant sections below based on your source and target versions.

19.1 Migrating from PostgreSQL 13

Key Changes Affecting Migration from PG 13:

- **PG 14 changes:** Multirange data types introduced. `password_encryption` default changed from `md5` to `scram-sha-256`. Clients using md5 passwords must be updated or pg_hba.conf must explicitly allow md5.
- **PG 14 changes:** `CURRENT_ROLE` now a reserved keyword. SQL referencing this as identifier will break.
- **PG 15 changes:** `PUBLIC` schema no longer has default CREATE permission for all users. Applications relying on any user creating objects in public schema will fail. Fix: `GRANT CREATE ON SCHEMA public TO PUBLIC;` after restore if needed.
- **PG 15 changes:** `python2` (plpython2u) language removed entirely. Must migrate to plpython3u.
- **PG 16 changes:** `postmaster` renamed to `postgres` as process name. Scripts checking process names must be updated.
- **PG 16 changes:** Many deprecated GUC parameters removed. Verify postgresql.conf compatibility before starting target.

19.2 Migrating from PostgreSQL 14

Key Changes Affecting Migration from PG 14:

- **PG 15 changes:** Public schema default CREATE permission removed (same as above). This is the most impactful change.
- **PG 15 changes:** `python2` language removed.
- **PG 16 changes:** Extended query protocol now used by `libpq` by default. Applications using custom protocol handling may be affected.
- **PG 16 changes:** `locale` provider 'builtin' introduced alongside 'libc' and 'icu'. No action needed but be aware for new databases.
- **PG 17 changes:** `pg_hba.conf` now supports `include` directives for modular configuration.

19.3 Migrating from PostgreSQL 15

Key Changes Affecting Migration from PG 15:

- **PG 16 changes:** Process renamed from `postmaster` to `postgres`.
- **PG 16 changes:** `createrole` no longer implies ability to grant membership. Review role permissions.
- **PG 17 changes:** `pg_dump` no longer dumps public schema ownership when it matches the default.
- **PG 17 changes:** JSON path operations now follow SQL/JSON standard more strictly.
- **PG 18 changes:** Review latest release notes before migration. Check extension compatibility with PG 18.

19.4 Common Post-Migration Tasks — All Version Upgrades

```
-- 1. Update extensions to latest version compatible with target PG:
\c your_database
ALTER EXTENSION extension_name UPDATE; -- repeat per extension per DB

-- 2. Fix public schema permissions if migrating from PG 13/14 to PG 15+:
-- Only if your application requires any user to create objects in public:
GRANT CREATE ON SCHEMA public TO PUBLIC;

-- 3. Run ANALYZE on all databases for fresh optimizer statistics:
vacuumdb --all --analyze-only --jobs=4 -U postgres

-- 4. Update password encryption for roles (if source used md5):
-- Check current encryption method:
SHOW password_encryption; -- should be 'scram-sha-256' on PG 14+
-- Re-set passwords to upgrade encryption:
ALTER ROLE username WITH PASSWORD 'same_password';

-- 5. Review and migrate postgresql.conf settings:
-- Compare source backup with target defaults:
diff /apps/backups/postgresql.conf.source_backup
/apps/pgsql_data/TARGET_VERSION/postgresql.conf

-- 6. Review and migrate pg_hba.conf:
diff /apps/backups/pg_hba.conf.source_backup
/apps/pgsql_data/TARGET_VERSION/pg_hba.conf

-- 7. Update connection strings in application configuration
-- if port or host has changed

-- 8. Verify cron jobs, monitoring, backup scripts point to new version paths
-- e.g., /apps/bin/ scripts, pgbackrest config, etc.
```

19.5 Migration Checklist Summary

Step	Action	When	Responsible
1	Pre-backup inventory (Section 17.1)	Before migration window	DBA
2	Install target PG version and required extensions	Before migration window	DBA / SysAdmin
3	Back up pg_hba.conf, postgresql.conf from source	Before migration window	DBA
4	Take pg_dumpall / pg_dump backup (Section 17)	Migration window start	DBA
5	Apply restore-tuned config on target (Sections 5–10)	Before restore	DBA
6	Restore to target cluster (Section 18)	Migration window	DBA
7	Post-restore verification (Section 18.5)	After restore	DBA + App Team
8	Fix public schema permissions if needed	After restore	DBA
9	Update extensions	After restore	DBA
10	ANALYZE all databases	After restore	DBA
11	Migrate pg_hba.conf and postgresql.conf	After restore	DBA
12	Set up streaming replication (Section 18.6)	After verification	DBA
13	Revert to production config (Section 12)	After replication setup	DBA
14	Application testing and validation	Before go-live	App Team
15	Switch application connection strings to new server	Go-live	App Team / DBA
16	Monitor for 24–48 hours post go-live	After go-live	DBA

20. Quick Reference Cheat Sheet

Order of Operations:

1. Back up /apps/pgsql_data/16/postgresql.auto.conf on BOTH servers
2. Check disk space on BOTH servers (Section 11 Step 3)
3. Apply memory/timeout params on SECONDARY → Restart SECONDARY
4. Verify SECONDARY is streaming and healthy
5. Apply ALL params on PRIMARY → Restart PRIMARY
6. Verify replication healthy on BOTH servers
7. Begin restore: pre-data → data → re-enable full_page_writes → post-data
8. Monitor replication lag, WAL size, slot health every 5–10 minutes
9. After restore: revert params, restart BOTH, run VACUUM ANALYZE

Key Safety Rules:

- **NEVER** leave autovacuum=off in production. Re-enable immediately after restore.
- **NEVER** leave full_page_writes=off in production. Risks data corruption on crash.
- **ALWAYS** back up `/apps/pgsql_data/16/postgresql.auto.conf` before making changes.
- **ALWAYS** apply Secondary changes BEFORE Primary changes.
- **ALWAYS** verify replication health between server restarts.
- **ALWAYS** monitor `/apps` filesystem — WAL, data, archives, backups, and logs share this space. Disk full kills the Primary.
- **ALWAYS** clean old archives in `/apps/pgsql_archives` and old backups in `/apps/backups` before starting large restores to maximize free space.
- For 700 GB+ restores, **STRONGLY consider dropping the replication slot** and rebuilding after.
- For 1 TB+ restores, **dropping the slot is the recommended approach**.

Critical Monitoring Queries:

```
-- Replication lag:
SELECT pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) FROM
pg_stat_replication;

-- Slot health:
SELECT slot_name, wal_status, conflicting FROM pg_replication_slots;

-- WAL directory size:
SELECT pg_size_pretty(sum(size)) FROM pg_ls_waldir();
```

```
-- /apps filesystem check (shared by WAL, data, archives, backups, logs):
df -h /apps
du -sh /apps/pgsql_data/16/pg_wal /apps/pgsql_archives /apps/backups /apps/logs

-- Checkpoint issues in log:
grep -i 'checkpoint.*too frequently' /apps/logs/postgresql-*.log
```

Key File Locations:

File/Directory	Path	Purpose
Main Filesystem	/apps	Root filesystem for all PostgreSQL directories
Data Directory	/apps/pgsql_data/16	PostgreSQL data files, config, WAL
Main Config	/apps/pgsql_data/16/postgresql.conf	Primary configuration file
Auto Config	/apps/pgsql_data/16/postgresql.auto.conf	ALTER SYSTEM overrides (takes precedence)
WAL Directory	/apps/pgsql_data/16/pg_wal	Write-Ahead Log segment files
Slot Metadata	/apps/pgsql_data/16/pg_replslot	Replication slot state files
WAL Archives	/apps/pgsql_archives	WAL archive destination for PITR
Server Logs	/apps/logs	PostgreSQL server log files
Binaries	/apps/bin	PostgreSQL binaries and utilities
Backups	/apps/backups	pgbackrest, pg_basebackup, pg_dumpall dumps
Config Backup	/apps/backups/postgresql.auto.conf.pre_restore_backup	Pre-restore config backup

pg_dumpall Migration Quick Reference:

```
-- ■■■ ALWAYS use TARGET version's pg_dumpall/pg_dump binary ■■■

-- Full cluster dump (plain SQL — for clusters < 200 GB):
/usr/pgsql-TARGET/bin/pg_dumpall -h SOURCE_IP -U postgres \
-f /apps/backups/full_dump.sql

-- Globals-only dump (roles + tablespaces):
/usr/pgsql-TARGET/bin/pg_dumpall -h SOURCE_IP -U postgres \
--globals-only -f /apps/backups/globals_only.sql

-- Per-database custom dump (for parallel restore):
/usr/pgsql-TARGET/bin/pg_dump -h SOURCE_IP -U postgres \
-Fc -d DB_NAME -f /apps/backups/DB_NAME.dump
```

```
-- Restore globals:
psql -U postgres -f /apps/backups/globals_only.sql

-- Parallel restore per database:
/usr/pgsql-TARGET/bin/pg_restore -U postgres -d DB_NAME \
--jobs=4 /apps/backups/DB_NAME.dump

-- Post-restore essentials:
vacuumdb --all --analyze-only --jobs=4 -U postgres
-- Fix public schema (if source was PG 13/14):
-- GRANT CREATE ON SCHEMA public TO PUBLIC;
```

Version Upgrade Path Quick Reference:

Source	Target	Binary Path	Critical Watch Items
PG 13	PG 16	/usr/pgsql-16/bin/	md5→scram-sha-256, public schema perms, python2 removal
PG 13	PG 17	/usr/pgsql-17/bin/	All PG 14+15+16 changes above, plus JSON path changes
PG 13	PG 18	/usr/pgsql-18/bin/	All cumulative changes. Thorough testing essential.
PG 14	PG 16	/usr/pgsql-16/bin/	public schema perms, postmaster→postgres rename
PG 14	PG 17	/usr/pgsql-17/bin/	public schema perms, createrole changes
PG 14	PG 18	/usr/pgsql-18/bin/	All cumulative. Check extensions compatibility.
PG 15	PG 16	/usr/pgsql-16/bin/	postmaster→postgres rename, createrole changes
PG 15	PG 17	/usr/pgsql-17/bin/	pg_dump public schema ownership change
PG 15	PG 18	/usr/pgsql-18/bin/	Review PG 18 release notes

END OF DOCUMENT

Review and update before each major database restore operation. Parameter values may need adjustment based on actual server hardware, available disk space, and observed behavior during previous restores.